

Lesson: Managing API Data Flow

Introduction

In modern web applications, handling client requests and server responses efficiently is crucial for delivering a seamless user experience. Express.js, a lightweight and flexible Node.js framework, simplifies this process by providing tools to manage request data, process it using middleware, and return structured responses. This lesson will guide you through the fundamentals of handling requests and responses in Express.js, including how to parse different types of data, use middleware effectively, and send well-structured JSON responses with appropriate status codes.

Learning Outcomes

By the end of this lesson, you will be able to:

1. Understand how requests and responses work in Express.js.
 2. Implement middleware to process requests and modify responses.
 3. Parse various types of request data, including JSON, query parameters, and headers.
 4. Use built-in, custom, and third-party middleware to enhance request handling.
 5. Send structured JSON responses with appropriate HTTP status codes.
-

Handling Requests and Responses in Express.js

When a client sends a request to an Express server, the server must extract and process the data sent by the client. This data can come in various forms:

- **Request Bodies:** Data sent in the body of the request (e.g., JSON or form data).
- **Query Parameters:** Key-value pairs appended to the URL for filtering, sorting, or pagination.
- **Headers:** Metadata about the request, such as authentication tokens or content type.

Each of these components serves a specific purpose and requires different handling techniques. Understanding how they interact is essential for building robust APIs.

Middleware in Express.js

Middleware is a fundamental concept in Express.js. Middleware functions execute during the lifecycle of a request to the server. They have access to three key objects:

1. **req (Request):** Contains information about the incoming HTTP request.
2. **res (Response):** Used to send responses back to the client.
3. **next:** A function that passes control to the next middleware or route handler.

MIDDLEWARES



Importance of Middleware

- **Execute Code:** Perform tasks like logging or validation.
- **Modify Request/Response Objects:** Add properties or change existing ones.
- **End the Request-Response Cycle:** Send a response directly to the client.
- **Pass Control:** Use `next()` to hand over control to the next middleware or route handler.

The order in which middleware is defined matters because it determines how requests are processed.

The Request-Response Cycle

Here's how a typical request flows through an Express server:

1. **Client Sends a Request:** The client makes an HTTP request (e.g., GET, POST) to the server.
2. **Middleware Processes the Request:** Middleware functions execute in the order they are defined.
3. **Router Handles the Request:** If the request matches a route, the corresponding route handler executes.
4. **Middleware Sends a Response:** Middleware or the route handler sends a response back to the client.

This structured flow ensures that every request is handled consistently and efficiently.

Types of Middleware

Built-In Middleware

Express provides several built-in middleware functions for common tasks:

- **`express.json()`:** Parses JSON request bodies. For example, if the client sends `{ "name": "John" }`, this middleware converts it into a JavaScript object (`{ name: "John" }`) and attaches it to `req.body`.
- **`express.urlencoded()`:** Parses URL-encoded data (common in HTML forms). For example, if the client sends `name=John&age=30`, this middleware converts it into `{ name: "John", age: "30" }`.

- **express.static()**: Serves static files like images, CSS, or JavaScript from a specified directory.

Custom Middleware

You can create your own middleware to perform tasks like logging, authentication, or validation. Here's an example:

```
// Logging Middleware
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`); // Logs the HTTP method and URL
  next(); // Passes control to the next middleware
};
app.use(logger);
```

Explanation: The logger middleware logs details about every incoming request (e.g., **GET /api/users**) before passing control to the next middleware.

Another example is authentication middleware:

```
// Authentication Middleware
const authenticate = (req, res, next) => {
  const token = req.headers.authorization; // Extracts the token from headers
  if (!token) {
    return res.status(401).json({ error: 'Unauthorized' }); // Sends a 401 response if no token is found
  }
  // Validate the token (Logic omitted for brevity)
  next();
};
app.get('/api/secure', authenticate, (req, res) => {
  res.json({ message: 'Access granted to secure resource' });
});
```

Explanation: This middleware checks if the client has provided an authentication token in the request headers. If not, it sends a **401 Unauthorized** response.

Third-Party Middleware

Express supports third-party middleware for common tasks. Some popular options include:

- **morgan**: Logs HTTP requests in a readable format.
- **cors**: Enables Cross-Origin Resource Sharing (CORS), allowing your API to be accessed from different domains.
- **helmet**: Secures your app by setting various HTTP headers.

Example with morgan:

```
const morgan = require('morgan');
app.use(morgan('dev')); // Logs requests in a developer-friendly format
```

Explanation: The `morgan` middleware logs details about each request, such as the HTTP method, URL, status code, and response time.

Parsing Request Data

Request Bodies

The request body contains data sent by the client, typically in the form of JSON, URL-encoded data, or multipart form data.

Parsing JSON Data Modern APIs often use JSON as the data format. To parse JSON data in Express, use the built-in `express.json()` middleware:

```
const express = require('express');
const app = express();

// Middleware to parse JSON request bodies
app.use(express.json());

app.post('/api/data', (req, res) => {
  console.log(req.body); // Logs the parsed JSON object
  res.send('Data received!');
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Explanation: If the client sends `{ "name": "John", "age": 30 }`, the `express.json()` middleware parses it into `{ name: "John", age: 30 }` and attaches it to `req.body`.

Parsing Multipart Form Data For file uploads or complex form submissions, you may encounter `multipart/form-data`. Use the `multer` library to handle this:

```
const multer = require('multer');
const upload = multer({ dest: 'uploads/' });

app.post('/upload', upload.single('file'), (req, res) => {
  console.log(req.file); // File details
  console.log(req.body); // Other form fields
  res.send('File uploaded!');
});
```

Explanation: The `upload.single('file')` middleware processes a single file upload. `req.file` contains details about the uploaded file, while `req.body` contains other form fields.

Query Parameters

Query parameters are key-value pairs appended to the URL after a `?`. They are commonly used for filtering, sorting, or pagination.

Example: Consider the following address:

```
https://example.com/products?category=electronics&sort=price_asc
```

Explanation:

- The base URL is `https://example.com/products`.
- The URL has two parameters:
 - `category` with a value of `electronics`
 - `sort` with a value of `price_asc`

Accessing Query Parameters

In Express, query parameters are automatically parsed and available in the `req.query` object:

```
app.get('/api/users', (req, res) => {  
  console.log(req.query); // { sort: 'asc', limit: '10' }  
  res.send(`Sorting by ${req.query.sort} and limiting to ${req.query.limit}  
  users.`);  
});
```

Use Cases:

- Filtering: `?status=active`
- Pagination: `?page=2&pageSize=20`
- Sorting: `?orderBy=name&direction=desc`

Headers

Headers provide metadata about the request, such as authentication tokens, content type, or custom information.

Accessing Headers Headers are accessible via the `req.headers` object:

```
app.get('/api/protected', (req, res) => {
  const authHeader = req.headers.authorization;
  if (!authHeader) {
    return res.status(401).send('Unauthorized');
  }
  console.log(authHeader); // e.g., "Bearer <token>"
  res.send('Access granted');
});
```

Common Headers:

- **Content-Type**: Indicates the format of the request body (e.g., `application/json`).
- **Authorization**: Used for passing authentication tokens.
- **Accept**: Specifies the response format the client expects.

Sending Structured JSON Responses

Once the server processes the request, it needs to send a response back to the client. A well-structured response ensures clarity and consistency in your API.

Sending JSON Responses

Express provides the `res.json()` method to send JSON responses. This method automatically sets the **Content-Type** header to `application/json`.

```
app.get('/api/user/:id', (req, res) => {
  const userId = req.params.id;
  const user = { id: userId, name: 'John Doe', email: 'john@example.com' };
  res.json(user);
});
```

Response Sent to Client:

```
{
  "id": "123",
  "name": "John Doe",
  "email": "john@example.com"
}
```

Response Status Codes

HTTP status codes indicate the result of the request. Common status codes include:

- **200 OK**: Successful request.
- **201 Created**: Resource successfully created.
- **400 Bad Request**: Invalid request data.

- **401 Unauthorized**: Authentication required.
- **404 Not Found**: Resource not found.
- **500 Internal Server Error**: Server-side error.

Example:

```
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;
  if (!username || !password) {
    return res.status(400).json({ error: 'Username and password are
required' });
  }
  // Authenticate user...
  res.status(200).json({ message: 'Login successful' });
});
```

Explanation: If the client doesn't provide a username or password, the server sends a **400 Bad Request** response. Otherwise, it sends a **200 OK** response.

Conclusion

Handling requests and responses effectively is a fundamental part of building Express.js applications. Middleware plays a key role in processing incoming requests, while structured responses improve API clarity. By mastering these concepts—middleware, request parsing, and response handling—you can create robust and scalable backend applications that efficiently manage client-server interactions.